

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development
Module 1
Practical –II

Roll no. : T019	Name : Sachin Ramesh Mehta
Class : TYIT	Batch : 1
Date of assignment : 17/07/2026	Date of assignment : 17/07/2026

Problem Solving by Searching (Uninformed Search)

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

Code :

```
from collections import deque

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()

def get_neighbors(state):
    neighbors = []

    blank_index = state.index(0)

    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, position_change in moves.items():
        new_index = blank_index + position_change

        # Check valid moves
        if move == "Up" and blank_index < 3:
            continue
        if move == "Down" and blank_index > 5:
            continue
        if move == "Left" and blank_index % 3 == 0:
            continue
        if move == "Right" and blank_index % 3 == 2:
            continue
```

```

        new_state = list(state)
        new_state[blank_index], new_state[new_index] = (
            new_state[new_index],
            new_state[blank_index]
        )

        neighbors.append((tuple(new_state), move))

    return neighbors

def bfs(initial_state, goal_state):
    queue = deque()

    queue.append((initial_state, []))

    visited = set()
    visited.add(initial_state)

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal_state:
            return path

        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [(move, neighbor)]))

    return None

initial_state = (
    1, 2, 3,
    4, 0, 5,
    6, 7, 8
)

goal_state = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("Initial State:")
print_state(initial_state)

print("Goal State:")
print_state(goal_state)

```

```

solution = bfs(initial_state, goal_state)

if solution:
    print("Shortest sequence of moves:")
    current_step = 1

    for move, state in solution:
        print("Step", current_step, "- Move:", move)
        print_state(state)
        current_step += 1

    print("Goal reached successfully.")
    print("Total number of moves:", len(solution))
else:
    print("No solution found.")
print("Sachin Mehta")

```

Output :

```

Initial State:
1 2 3
4 0 5
6 7 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest sequence of moves:
Step 1 - Move: Right
1 2 3
4 5 0
6 7 8

Step 2 - Move: Down
1 2 3
4 5 8
6 7 0

Step 3 - Move: Left
1 2 3
4 5 8
6 0 7
...

Goal reached successfully.
Total number of moves: 14
Sachin Mehta

```

2. Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

Code :

```

print("Sachin Mehta")

```

```

j1 = int(input("Enter capacity of Jug 1: "))
j2 = int(input("Enter capacity of Jug 2: "))
goal = int(input("Enter target amount: "))

visited = set()

def dfs(x, y, path):

    if x == goal or y == goal:
        path.append((x, y))
        print("\nGoal Achieved Successfully!")
        print("Solution Path:")
        for state in path:
            print(state)
        return True

    if (x, y) in visited:
        return False

    visited.add((x, y))
    path.append((x, y))

    next_states = [
        (j1, y),
        (x, j2),
        (0, y),
        (x, 0),

        (
            x - min(x, j2 - y),
            y + min(x, j2 - y)
        ),

        (
            x + min(y, j1 - x),
            y - min(y, j1 - x)
        )
    ]

    for state in next_states:
        if dfs(state[0], state[1], path.copy()):
            return True

    return False

if not dfs(0, 0, []):
    print("No solution found")

```

Output :

Sachin Mehta

Goal Achieved Successfully!

Solution Path:

(0, 0)

(4, 0)

(4, 3)

(0, 3)

(3, 0)

(3, 3)

(4, 2)

3. Given a weighted graph representing cities and distances between them, write a

Python program to find the least-cost path from Arad to Bucharest using Uniform Cost

Search (UCS). [Provide any other weighted graph for applying UCS]

Code :

```
import heapq
```

```
graph = {  
    'Mumbai': [('Pune', 150), ('Hyderabad', 710)],  
    'Pune': [('Mumbai', 150), ('Bangalore', 840)],  
    'Hyderabad': [('Mumbai', 710), ('Bangalore', 570)],  
    'Bangalore': []  
}
```

```
def ucs(start, goal):
```

```
    queue = [(0, start, [start])]
```

```
    visited = []
```

```
    while queue:
```

```
        cost, city, path = heapq.heappop(queue)
```

```
        if city == goal:
```

```
            return cost, path
```

```
        if city not in visited:
```

```
            visited.append(city)
```

```
            for neighbor, distance in graph[city]:
```

```
                heapq.heappush(  
                    queue,  
                    (  
                        cost + distance,
```

```
                        path + [neighbor],
```

```
                        cost + distance,
```

```
        neighbor,  
        path + [neighbor]  
    )  
)
```

```
cost, path = ucs("Mumbai", "Bangalore")  
print("Least-cost path:")  
print("Path:", " -> ".join(path))  
print("Total Distance:", cost, "km")  
print("Sachin Mehta")
```

Output :

```
Least-cost path:  
Path: Mumbai -> Pune -> Bangalore  
Total Distance: 990 km  
Sachin Mehta
```